

Arrays_Question:

Arrays Problems:

1. Reverse an Array
 2. Find the Maximum & Minimum Element
 3. Find the Sum of Elements
 4. Find the Second Largest Element
 5. Count Frequency of Elements
 6. Check if Array is Sorted
 7. Rotate Array by k Positions: Rotate the array to the right by k positions.
 8. Find Pair with Given Sum: Find a pair of elements that adds up to a target sum.
 9. Remove Duplicates from Array: Remove duplicates from the array while maintaining order.
 10. Merge Two Sorted Arrays
 11. Remove given Element from Array
 12. Find the Missing Number: Find the missing number in an array of size n containing numbers from 1 to n.
 13. Find Duplicates in an Array
 14. Find Intersection of Two Arrays: Find the common elements between two arrays.
 15. Find Union of Two Arrays
 16. Check if Two Arrays Are Equal: if two arrays contain the same elements
 17. Find the Leader Elements: An element is a leader if it is greater than all elements to its right.
 18. Move Zeroes to End: Move all zeroes in an array to the end while maintaining the order of non-zero elements.
 19. Find Subarray with Given Sum.
 20. Rotate Array to the Left by k Positions
 21. Find the Kth Smallest Element
 22. Find All Subarrays
 23. Maximum Sum Subarray (Kadane's Algorithm)
 24. Rearrange Array Alternately: Rearrange an array such that elements alternate between the largest and smallest.
 25. Find Majority Element: Find the element that appears more than n/2 times.
 26. Find Peak Element: A peak element is greater than its neighbors. Find one such element.
 27. Find the First Missing Positive: Find the smallest positive integer missing in the array.
 28. Sort an Array of 0s, 1s, and 2s: Sort an array consisting of only 0s, 1s, and 2s.
 29. Find the Longest Consecutive Sequence: Find the length of the longest consecutive sequence of integers.
 30. Product of Array Except Self
Given an array, return a new array where each element is the product of all elements except itself.
Do not use division.
- Input: [1,2,3,4]
Output: [24,12,8,6]
31. Find Equilibrium Index: Find an index such that sum of elements on left = sum on right.
 32. Find Maximum Product Pair: Find two elements whose product is maximum.
 33. Find Maximum Difference (j > i)

1. Reverse an array.

<https://leetcode.com/problems/reverse-string/>

```
class Solution:
    def reverseString(self, s: List[str]) -> None:
        return s.reverse()
```

2. Find the maximum and minimum elements.

<https://leetcode.com/problems/minimum-initial-energy-to-finish-tasks/>

```
class Solution:
    def maxAndMin(self, nums: list[int]) -> tuple(int,int):
        minn = min(nums)
        maxx = max(nums)
        return minn,maxx
```

3. Find the sum of elements:

<https://leetcode.com/problems/running-sum-of-1d-array/>

```
class Solution:
    def SumOfArray(self, nums: list[int]): -> List[int]:
        lst = []
        sum = 0
        for i in nums:
            sum = sum+i
            lst.append(sum)
        return lst
```

4. Find the second largest number:

<https://leetcode.com/problems/third-maximum-number/>

```
class Solution:
    def secondLargest(self, nums: List[int]): -> int:
        s = set(nums)
        lst = sorted(s, reverse=True)

        if len(lst) >= 3:
            return lst[1]
        else:
            return lst[0]
```

5. Count frequency of element:

<https://leetcode.com/problems/top-k-frequent-elements/>

```
class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        count = {}
        freq = [[] for i in range(len(nums)+1)]

        for n in nums:
            count[n] = 1 + count.get(n,0)
        for n,c in count.items():
```

```

        freq[c].append(n)
    res = []
    for i in range(len(freq)-1, 0, -1):
        for n in freq:
            res.append(n)
            if len(res) == k:
                return res

```

6. Check if the array is sorted

<https://leetcode.com/problems/check-if-array-is-sorted-and-rotated/>

```

class Solution:
    def check(self, nums: List[int]) -> bool:
        N = len(nums)
        count = 1
        for i in range(1, 2*N):
            if nums[(i - 1) % N] <= nums[i % N]:
                count += 1
            else:
                count = 1
            if count == N:
                return True
        return False

```

7. Rotate the array by k positions: Rotate the array to the right by k positions.

<https://leetcode.com/problems/rotate-array/>

```

class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        k = k % len(nums)
        l, r = 0, len(nums)
        while l < r:
            nums[l], nums[r] = nums[r], nums[l]
            l = l + 1
            r = r - 1
        l = k
        r = k - 1

```

```

while(l < r):
    nums[l],nums[r] = nums[r],nums[l]
    l = l+1
    r = r-1

l = k
r = len(nums)
while(l < r):
    nums[l],nums[r] = nums[r],nums[l]
    l = l+1
    r = r-1

```

8. Find pairs with given sum: Find a pair of elements that can add up to the sum.

<https://leetcode.com/problems/two-sum/>

```

class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        pair_idx = {}

        for i, num in enumerate(nums):
            if target - num in pair_idx:
                return [i, pair_idx[target - num]]
            pair_idx[num] = i

```

9. Remove Duplicates from an array.

<https://leetcode.com/problems/remove-duplicates-from-sorted-array/>

```

class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:

        l = 1
        for r in range(1,len(nums)):
            if nums[r] != nums[r-1]:
                nums[l] = nums[r]

```

```
        l += 1
    return l
```

10. Merge two sorted arrays.

<https://leetcode.com/problems/merge-sorted-array/>

```
class Solution:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        for i in nums2:
            nums1.append(i)

        del nums1[m:m+n]
        nums1.sort()
        return nums1
```

11. Remove the given element from the array.

<https://leetcode.com/problems/remove-element/>

```
class Solution:
    def removeElement(self, nums: List[int], val: int) -> int:
        k = 0
        for i in range(len(nums)):
            if nums[i] != val:
                nums[k] = nums[i]
                k=k+1
        return k
```

12. Find the missing number:

<https://leetcode.com/problems/missing-number/>

Time = O(n) and space = O(n)

```
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        s = set(nums)
```

```

for i in range(len(nums)+1):
    if i not in s:
        return i

```

13. Find all duplicates in the array:

<https://leetcode.com/problems/find-all-duplicates-in-an-array/>

Time = $O(n)$ and space = $O(n)$

```

class Solution:
    def findDuplicates(self, nums: List[int]) -> List[int]:
        lst = []
        s = set()
        for i in nums:
            if i not in s:
                s.add(i)
            else:
                lst.append(i)
        return lst

```

14. Find the intersection of two arrays: find the common elements between two arrays.

<https://leetcode.com/problems/intersection-of-two-arrays/>

```

class Solution:
    def intersection(self, nums1: List[int], nums2: List[int]) -> List[int]:
        seen = set(nums1)
        res = []
        for n in nums2:
            if n in seen:
                res.append(n)
                seen.remove(n)
        return res

```

15. Find the union of two arrays

<https://leetcode.com/problems/union-of-two-arrays/>

```
class Solution:
    def FindUnion(self, arr1, arr2, n,m):
        freq = {}
        for i in range(n):
            freq[arr1[i]] = freq.get(arr1[i],0) + 1
        for i in range(m):
            freq[arr2[i]] = freq.get(arr2[i],0) + 1
        Union = sorted(freq.keys())

        return Union
if __name__=="__main__":
    n = 10
```

16. Check if the two arrays are equal: if they contain the same elements.

<https://leetcode.com/problems/valid-anagram/>

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False
        countS, countT = {}, {}

        for i in range(len(s)):
            countS[s[i]] = 1 + countS.get(s[i],0)
            countT[t[i]] = 1 + countT.get(t[i],0)
        for c in countS:
            if countS[c] != countT.get(c,0):
                return False
        return True
```

17. Find the leader elements: An element is a leader if it is greater than all elements to its right.

<https://leetcode.com/problems/find-leaders-in-an-array/>

```

def leaders(arr):
    result = []
    n = len(arr)
    for i in range(n):
        for j in range(i+1, n):
            if arr[i] < arr[j]:
                break
        else:
            result.append(arr[i])
    return result

if __name__ == "__main__":
    arr = [16,15,34,7,8,5,4,9,3]
    result = leaders(arr)
    print(" ".join(map(str,result)))

```

18. Move zeros to the end: Move all zeros in an array to the end while maintaining the order of non-zero elements.

<https://leetcode.com/problems/move-zeros/>

Time = $O(n)$ and space = $O(1)$

```

class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        j = 0
        for i in range(len(nums)):
            if nums[i] != 0:
                nums[j], nums[i] = nums[i], nums[j]
                j += 1
        return nums

```

19. Find the subarray with the given sum.

<https://leetcode.com/problems/subarray-sum-equals-k/>

```

class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        res = 0
        curSum = 0

```



```

pre = { 0:1 }

for n in nums:
    curSum += n
    deff = curSum - k

    res += pre.get(diff,0)
    pre[curSum] = 1 + pre.get(curSum,0)
return res

```

20. Rotate the array to the left by k positions.

<https://leetcode.com/problems/rotate-array/>

```

class Solution:
    def rotate(self, nums: List[int], k:int) -> None:
        k = k % len(nums)
        l,r = 0,len(nums)
        while l < r:
            nums[l], nums[r] = nums[r], nums[l]
            l = l + 1
            r = r - 1
        l = k
        r = k - 1
        while(l < r):
            nums[l],nums[r] = nums[r],nums[l]
            l = l+1
            r = r-1

        l = k
        r = len(nums)
        while(l < r):
            nums[l],nums[r] = nums[r],nums[l]
            l = l+1
            r = r-1

```

21. Find the kth largest element.

<https://leetcode.com/problems/kth-smallest-element-in-a-sorted-matrix/description/>

```
class Solution:
    def findKthLargest(self, nums: List[int], k:int) -> int:
        min_heap = []
        for num in nums:
            if len(min_heap) < k:
                heapq.heappush(min_heap, num)
            else:
                heapq.heappushpop(min_heap, num)
        return min_heap[0]
```

22. Find all subarrays.

<https://leetcode.com/problems/count-complete-subarrays-in-an-array/description/>

```
class Solution:
    def countCompleteSubarrays(self, nums: List[int]) -> int:
        N = len(nums)
        C = len(set(nums))
        count = 0

        for i in range(N):
            s = set()
            for j in range(i, N):
                s.add(nums[j])

                if len(s) == C:
                    count += 1
                elif len(s) > C:
                    break
        return count
```

23. Maximum Subarray problem (Kadane's algorithm).

<https://leetcode.com/problems/maximum-subarray/>

Time = $O(n)$ and $O(1)$

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        curSum = 0
        maxSum = min(nums)
        for i in nums:
            curSum += i
            maxSum = max(curSum, maxSum)
            if (curSum < 0):
                curSum = 0
        return maxSum
```

24. Rearrange the array alternatively.

<https://leetcode.com/problems/rearrange-array-elements-by-sign/description/>

```
class Solution:
    def rearrangeArray(self, nums: List[int]) -> List[int]:
        i, j = 0, 1
        res = [0] * len(nums)
        for k in range(len(nums)):
            if nums[k] > 0:
                res[i] = nums[k]
            else:
                res[j] = nums[k]
                j += 2
        return res
```

25. Find the majority element: find the element that appears more than $n/2$ times

<https://leetcode.com/problems/majority-element/description/>

```
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        ans = -1
```

```

count = 0
for num in nums:
    if count == 0:
        ans = num
    if ans == num:
        count += 1
    else:
        count -= 1
return ans

```

26. Find the peak element:

<https://leetcode.com/problems/find-peak-element/>

```

class Solution:
    def findPeakElement(self, nums: List[int]) -> int:
        l, r = 0, len(nums) - 1
        while l <= r:
            m = l + ((r - l) // 2)
            if m > 0 and nums[m] < nums[m - 1]:
                r = m - 1
            elif m < len(nums) - 1 and nums[m] < nums[m + 1]:
                l = m + 1
            else:
                return m

```

27. Find the first missing positive.

<https://leetcode.com/problems/first-missing-positive/>

It is a hard level problem !!

28. Sort color problem similar to sort 1s, 2s, 3s array .

<https://leetcode.com/problems/sort-colors/>

```

class Solution:
    def sortColor(self, nums: List[int]) -> None:
        counts = [0, 0, 0]

```

```

for color in nums:
    counts[color] += 1
R,W,B = counts
nums[:R] = [0]*R
nums[R:R+W] = [1] * W
nums[R+W:] = [2] * B

```

29. Find the longest Consecutive Sequence:

<https://leetcode.com/problems/longest-consecutive-sequence/>

```

class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        numSet = set(nums)
        longest = 0

        for n in nums:
            if (n - 1) not in numSet:
                length = 0
                while (n + length) in numSet:
                    length += 1
                longest = max(length, longest)
        return longest

```

30. Product of array except self

<https://leetcode.com/problems/product-of-array-except-self/>

```

class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        res = [1] * (len(nums))

        pre = 1
        for i in range(len(nums)):
            res[i] = pre
            pre *= nums[i]
        postfix = 1
        for i in range(len(nums) - 1, -1, -1):
            res[i] *= postfix

```

```
        postfix *= nums[i]
    return res
```

31. Find equilibrium Index: Find an index such that the sum of elements of the left = the sum on the right.

<https://leetcode.com/problems/find-pivot-index/>

```
class Solution:
    def pivotIndex(self, nums: List[int]) -> int:
        total = sum(nums)
        leftSum = 0
        for i in range(len(nums)):
            rightSum = total - nums[i] - leftSum
            if leftSum == rightSum:
                return i
            leftSum += nums[i]
        return -1
```

32. Find the maximum product pair: Find two elements whose product is maximum.

<https://leetcode.com/problems/find-pivot-index/>

```
class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        nums.sort(reverse=True)
        return (nums[0]-1)*(nums[1]-1)
```

33. Find the maximum difference (j < i).

<https://leetcode.com/problems/best-time-to-buy-and-sell-stock/>

```
class Solution:
    def MaxProfit(self, prices: List[int]) -> int:
        l = 0
        r = 1
        maxProfit = 0
```

```
while( r < len(prices):  
    if prices[l] < prices[r]:  
        profit = prices[r] - prices[l]  
        maxProfit = max(maxProfit, profit)  
    else:  
        l = r  
    r += 1  
return maxProfit
```